Lecture 5

I/O Organization

Previously ..

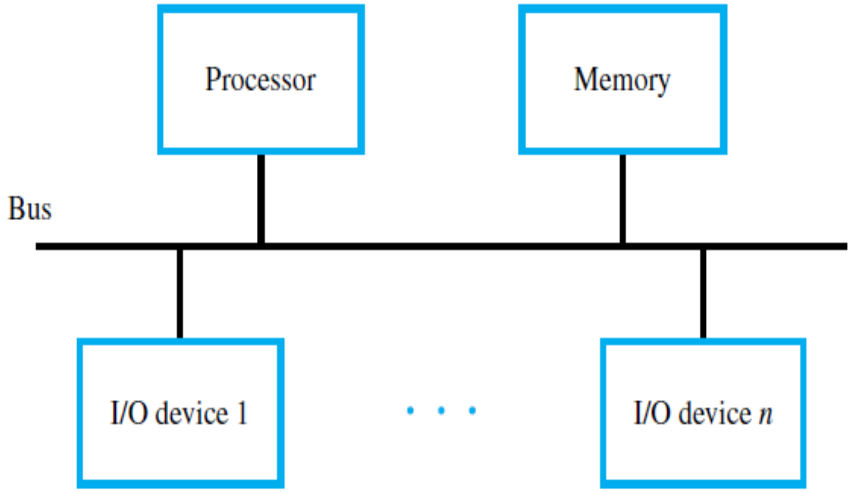
- Assembly Programming
  - Assembly Instruction
  - Assembly Directives
  - Stacks
    - Push and Pop
  - Subroutines
    - Nesting
    - Parameter Passing

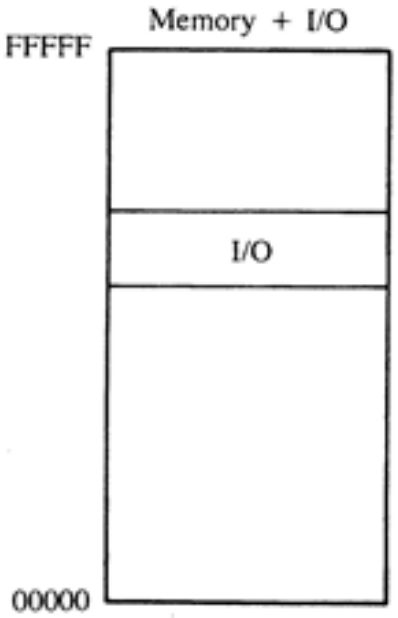
Basic I/O

(Chapter 3)

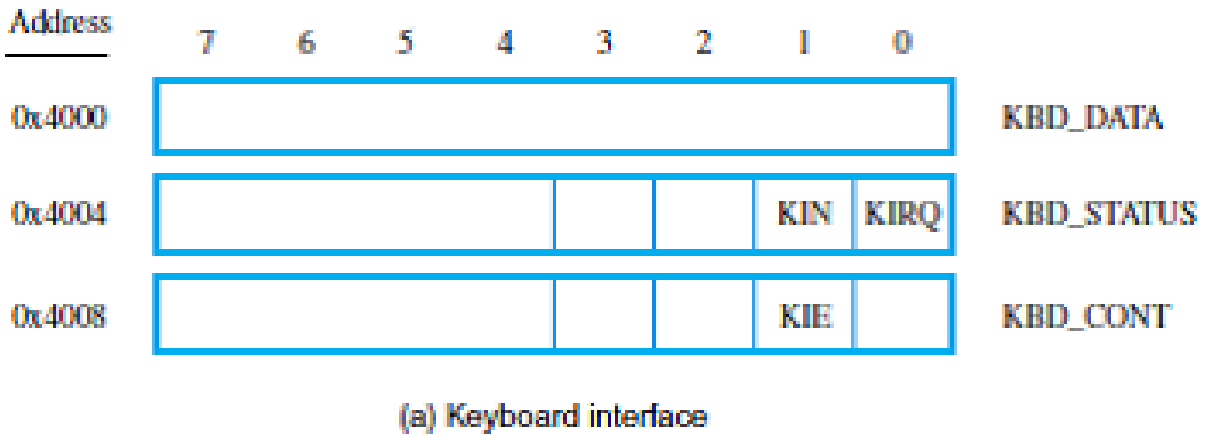
Accessing I/O Devices

- I/O device must appear to the processor as consisting of some addressable locations, **just like the memory**.
- The I/O devices and the memory share the same address space, this arrangement is called ***memory-mapped I/O.***
- Single bus structure is connected to I/O device, at ***device interface***.

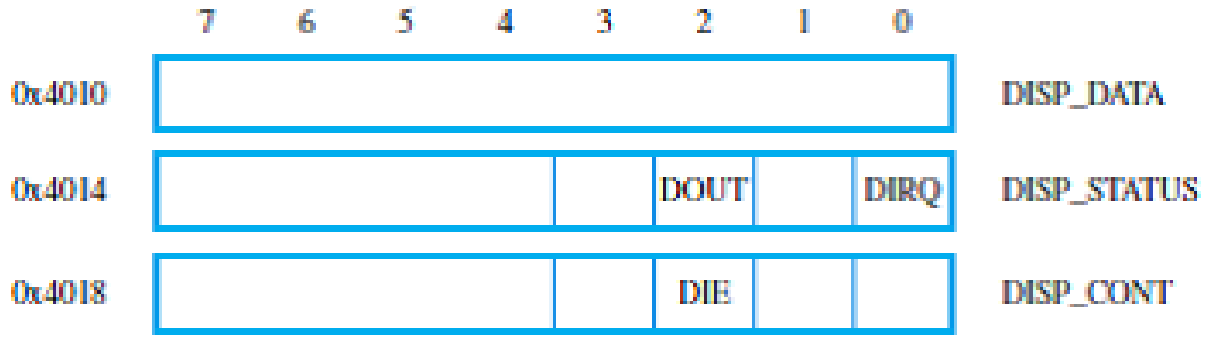




Device Interface Registers



(a) Keyboard interface



(b) Display interface

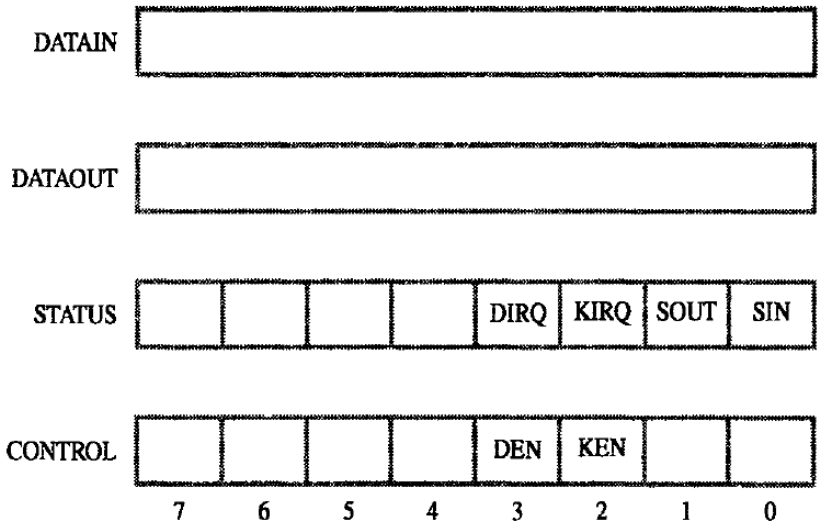
Program-controlled I/O

```
READ    TestBit     #3,INSTATUS
        Branch=0    READ
        Move        DATAIN,(R0)

ECHO    TestBit     #3,OUTSTATUS
        Branch=0    ECHO
        Move        (R0),DATAOUT
```

- A task that read characters from a keyboard, stores these data in the memory, and display the same characters on the screen, called ***program-controlled I/O***.
- The difference speed of human typing, I/O devices, and the processor, needed the mechanisms to synchronize.
  - A signaling protocol or ***polling method***
  - **I/O Interrupt** or hardware interrupt

```
        Move        #LINE,R0          Initialize memory pointer.
WAITK   TestBit     #0,STATUS         Test SIN.
        Branch=0    WAITK             Wait for character to be entered.
        Move        DATAIN,R1         Read character.
WAITD   TestBit     #1,STATUS         Test SOUT.
        Branch=0    WAITD             Wait for display to become ready.
        Move        R1,DATAOUT        Send character to display.
        Move        R1,(R0)+          Store charater and advance pointer.
        Compare     #$0D,R1           Check if Carriage Return.
        Branch≠0    WAITK             If not, get another character.
        Move        #$0A,DATAOUT      Otherwise, send Line Feed.
        Call        PROCESS           Call a subroutine to process the
                                        the input line.
```



- This processor repeatedly checks a status flags to achieve the required synchronization between the processor and an input or output device
- Also called **Processor Polls the device**
- Other methods are Interrupt and DMA

Hardware (I/O) Interrupt

- Polling, usually spend a lot of time in wait loops. During this period, a processor is not performing any useful computation.
- A processor is thus, often arranged to do something else during the waiting periods. An I/O device is capable of alerting a processor when it is done by sending a hardware signal called ***an interrupt***.
- There is a bus line that is dedicated for interrupts, called ***an interrupt-request (IRQ)*** line.

Interrupt Example

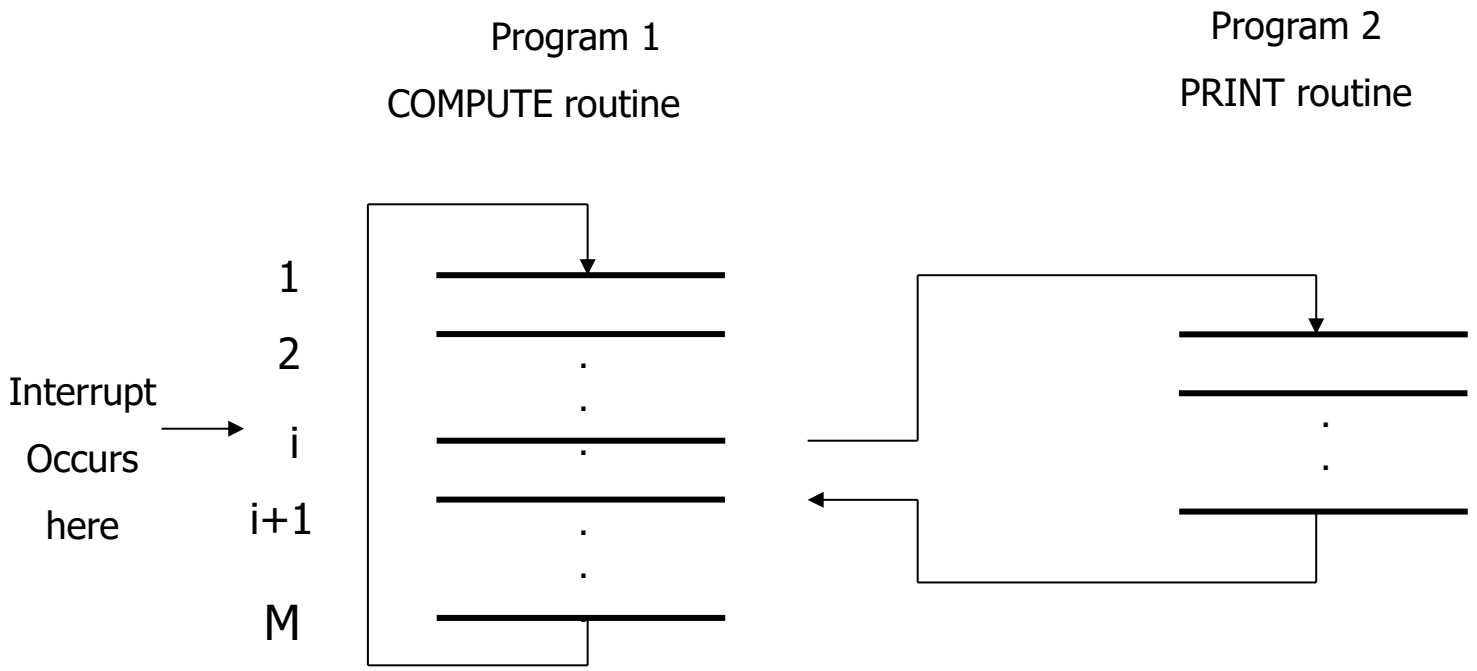
- EX Process one line of data, and then send the result to a printer
- Overlap computation and printing
  - During printing a line, a processor goes back to perform a useful computation. When the printer is ready for the next line, an interrupt is sent via an interrupted-request line.

Interrupt Service Routine (ISR)

- The routine executed in response to an interrupt request is called the ***interrupt-service routine***.
- Processor responds or interrupt request responds to interrupted device by sending the control signal called **Interrupt Acknowledge**.
- The ISR returns to interrupted program **using Return-from-Interrupt instruction**.

ISR Example

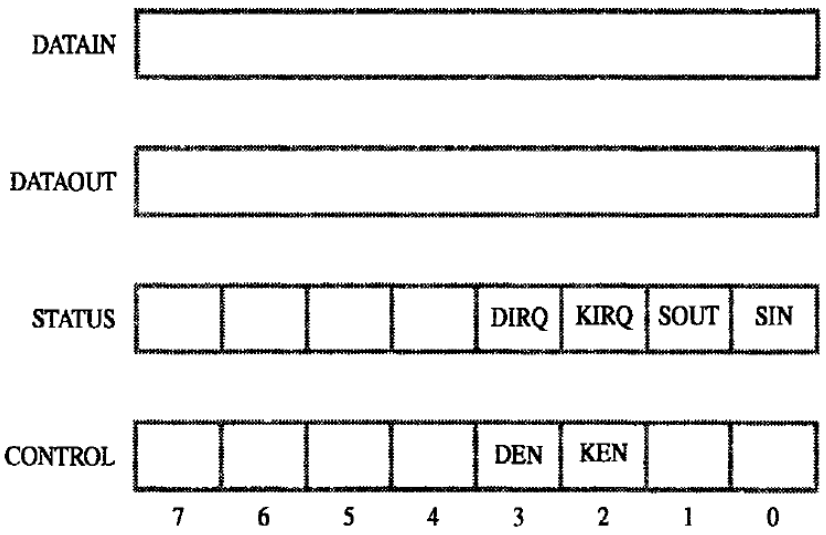
EX A processor receives an interrupt during the execution of instruction i. i will be executed until finish, then the PC is loaded with the address of the first instruction of the interrupt-service routine. i+1 is stored in a known location (link register or stack, maybe). When the interrupt service is done, i+1 address is put back to the PC.



Interrupt Latency

- ***Interrupt latency*** : the time between interrupt request is received and the time when the service routine starts execution.
- The step of saving and restoring registers involves memory transfers that increase the total execution time, and hence represent execution overhead.
- The task of saving and restoring information can be done automatically by the processor or by program instructions.
- This kind of delay not acceptable in **Real-time processing**.

Enabling and Disabling Interrupt (1)



- Keyboard interrupt enable bit KEN
- Display interrupt enable bit DEN
- KEN and/or DEN = 1 then the devices can interrupt the processor
- KIRQ and/or DIRQ = 1 indicate that both devices are requesting an interrupt.

Enabling and Disabling Interrupt (2)

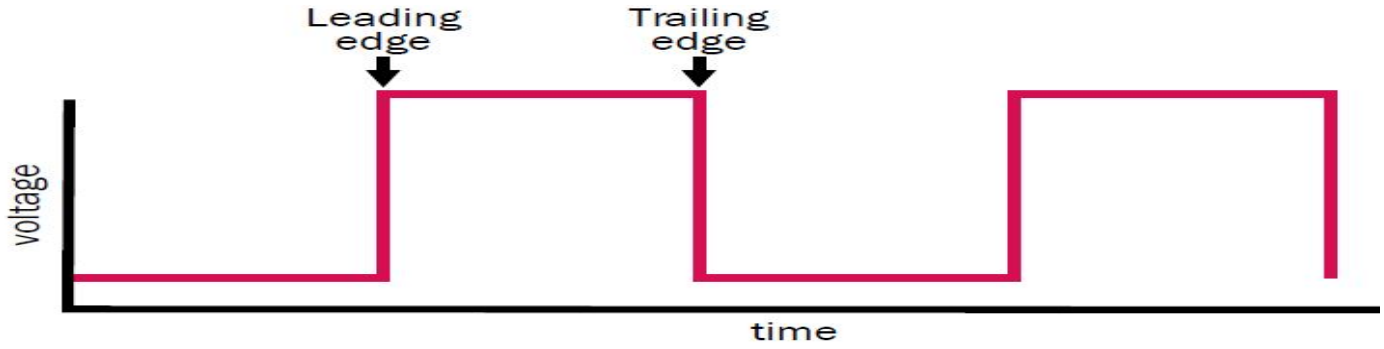
- There are cases where interrupt should be ignored. Ex. Don't service a print-routine, if there is no output to be printed.
- To prevent an interrupt to occur on top of another interrupt from the same device, which could lead to an infinite loop (a system cannot recover).
- Interrupt can be enabled or disabled by a programmer. A simple way is to provide machine instructions, such as interrupt-enable (IE=1) and interrupt-disable (IE=0).

Solution for Successive Interruption (1)

- First
  - Ignore the IRQ until complete the current ISR
  - i.e. first instruction of ISR is Interrupt Disable (IE=0) and last instruction is Interrupt Enable (IE=1)
- Second
  - Processor first saves the contents of the program counter (PC) and the processor status (PS) register with IE=1 on stack and automatically disable interrupts (IE=0) before starting the execution of the interrupt-service routine.
  - When return from interrupt instruction is executed the contents of PC and PS is popped with IE=1 from stack.

Solution for Successive Interruption (2)

- Third
  - Add the interrupt-handling circuit in the processor.
  - IRQ line must accepts only leading edge of the signal (Edge triggered line)
  - Processor receives only one request regardless of how long the line is activated.
  - So no question of successive interruption or no need of explicit instruction for enable/disable interrupt.



Interrupt for Multiple Devices

- A processor needs to be able to determine which device sends an interrupt.
- If two devices send interrupt signals at the same time, the processor needs to know which one should be serviced first?
- Should a device be allowed to interrupt the processor while another interrupt is being serviced?
- Given that different devices are likely to require different interrupt-service routines. How can the processor obtain the starting address of the appropriate routine in each case?

Interrupt Polling

- To identify an interrupt device, a ***Interrupt polling scheme*** can be used, where the special ISR polls all the I/O devices connected to the bus. The first device with the IRQ bit set is then serviced.
- Interrupt bit is in the device status-register, KIRQ and DIRQ. Device request for interrupt the IRQ=1.
- Again this technique, the processor serve the device using **polling**. This technique is time consuming.

Interrupt Vector

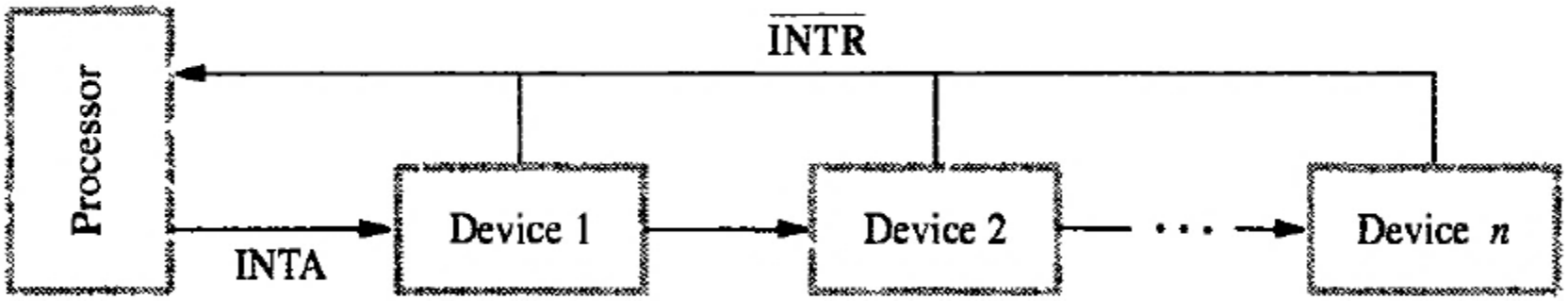
- A device can identify itself by sending a special code via a bus to a processor. The code can be the starting address of the service routine (interrupt from a given device must start at the same location)
- This address is called ***interrupt vector***
- Commonly, this technique is to allocate permanently an area in the memory to hold the addresses of interrupt-service routines. These addresses are usually referred to as interrupt vectors, and they are said to constitute **the interrupt-vector table (IVT)**.

Simultaneous Requests

- If the interrupt polling scheme is used, a priority could be assigned from the order of polled devices, ordering by processor.
- If the interrupt vector scheme is used, devices can be connected in a **daisy chain**.
  - Devices that are closer to the processor will have a higher priority. When an interrupt is raised, a process sends out a signal to the first device. If the device wants a service, it will take the signal. Otherwise, it will pass the signal to the next device in the chain.

Implement of Simultaneous Requests

- **Daisy Chain**



- INTR is common to all device
- INTA connected in daisy chain fashion where INTA signal propagate serially through the devices.

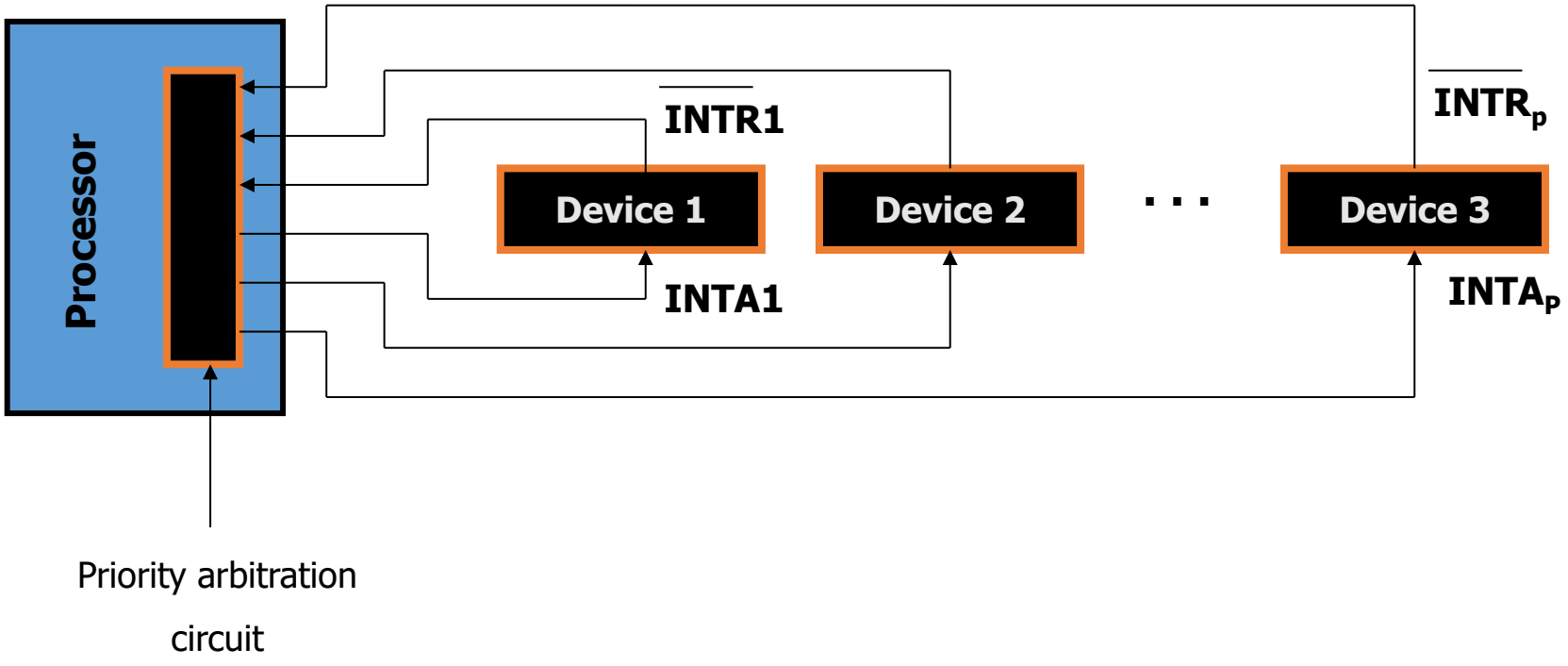
Interrupt Nesting

- It is sometime necessary to accept an interrupt while servicing another interrupt from another device.
- Ex. A real time clock need a processor every so often to increment counters the keep track of time.
- Thus, devices should be organized in a priority structure. Only an interrupt from a higher-priority device will be accepted when a processor is servicing another interrupt.

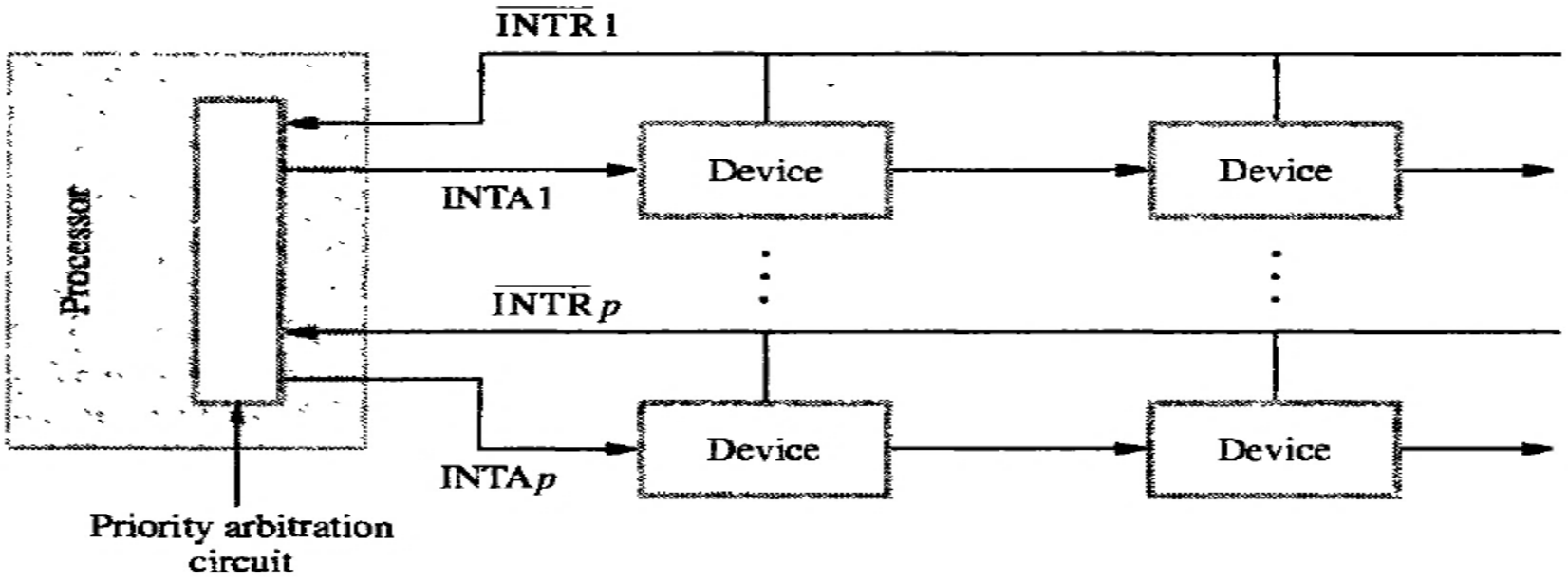
Interrupt Priority

- The processor's priority can be encoded in a few bits of the processor status register
- By using a separate INTR and INTA line for each device. Each individual line is assigned a different priority level.
- All IRQs are sent to **a priority arbitration circuit**.
- Only a higher-priority request is accepted during another interrupt service.

Implementation of Interrupt Priority



Interrupt Priority + Daisy Chain



Exceptions: Recovery from Errors

- Many computers include an error-checking code in the main memory, which allows detection of errors in the stored data. If an error occurs, the control hardware detects it and informs the processor by raising an interrupt.
- The processor proceeds in exactly the same manner as in the case of an I/O interrupt request.
- It suspends the program being executed and starts an exception-service routine, which takes appropriate action to recover from the error, if possible or to inform the user about it.

Exceptions: Debugging

- System software usually includes a program called a debugger, which helps the programmer find errors in a program.
- The debugger uses exceptions to provide two important facilities: trace mode and breakpoints.

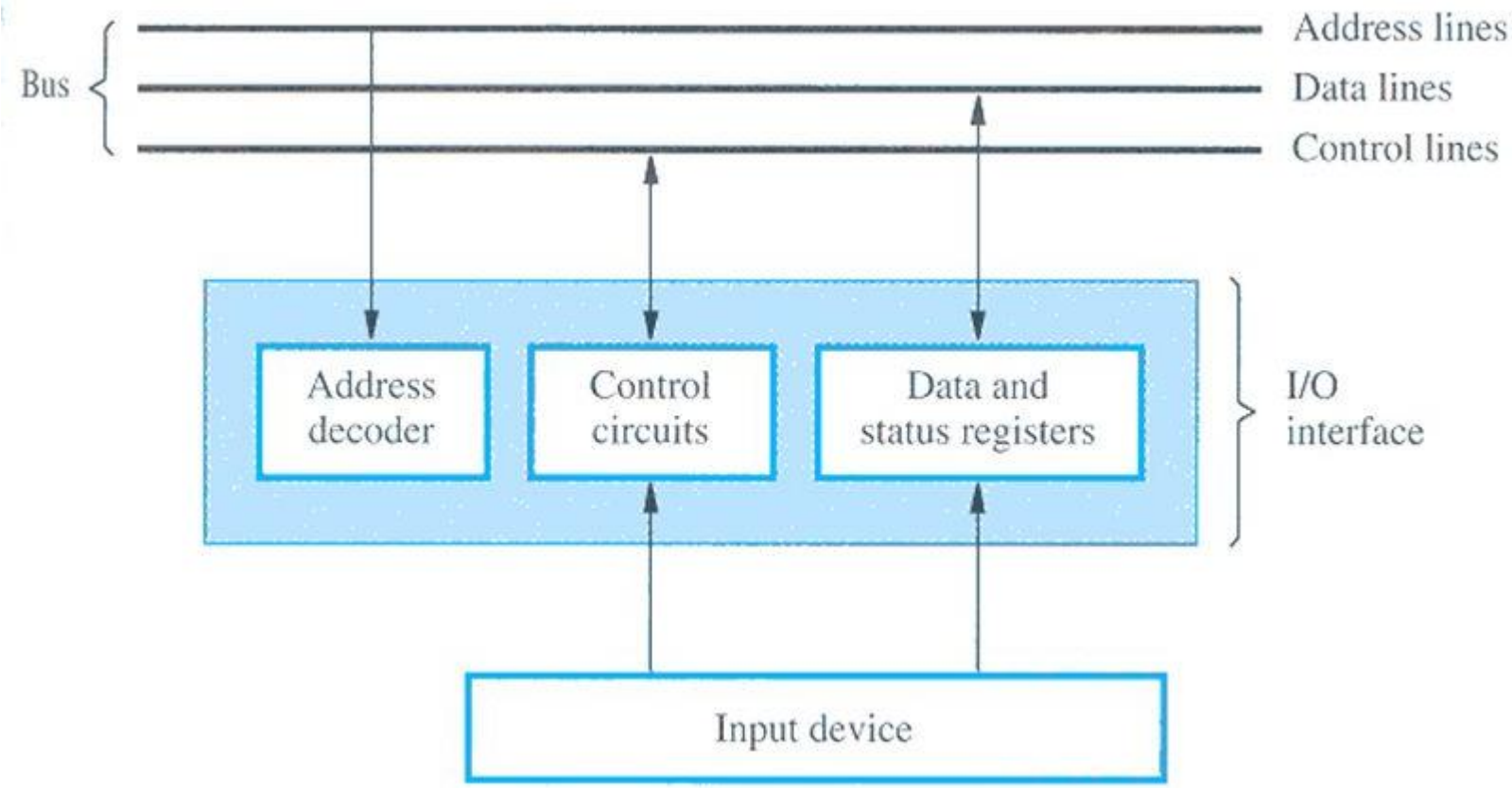
Exceptions: Privilege Exception

- To protect the OS from being corrupted by user programs certain instructions (Privilege Instructions) can executed only when processor is in superior mode.
- In user mode these instruction not executed, so that user program does not change the priority level of the processor or program access area.

I/O Organization

(Chapter 7)

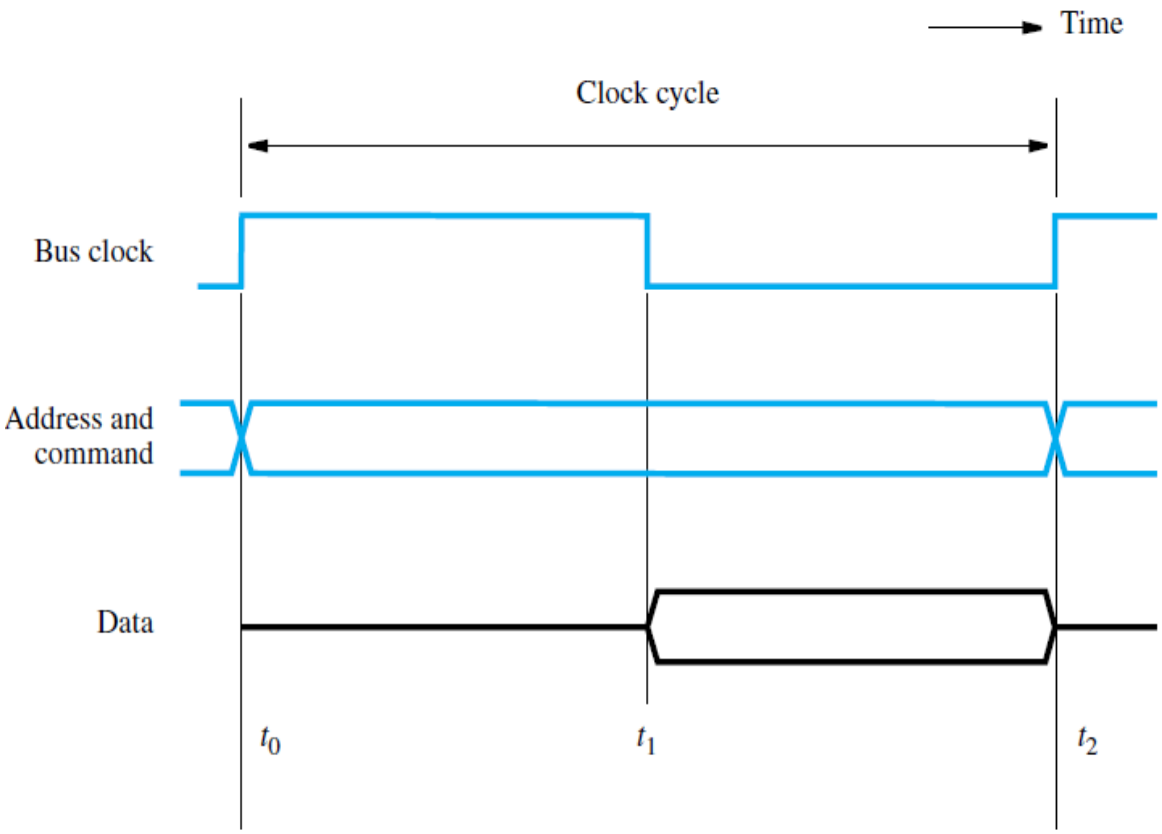
Bus Structure



Bus Operation

- Various devices use the bus.
- The ***bus protocol*** determines when a device may place information on the bus, when it may load the data on the bus into one of its registers, and so on.
- These rules are implemented by control signals that indicate what and when actions are to be taken.
- **Master or initiator- Device** which initiates data transfers.
- **Slave or target- Device** is addressed by master.

Synchronous Bus: Idealized Timing Diagram (1)



- All devices derive timing information from a control line called the ***bus clock***
- The timing diagram shows an **idealized representation** of the actions that take place on the bus lines.

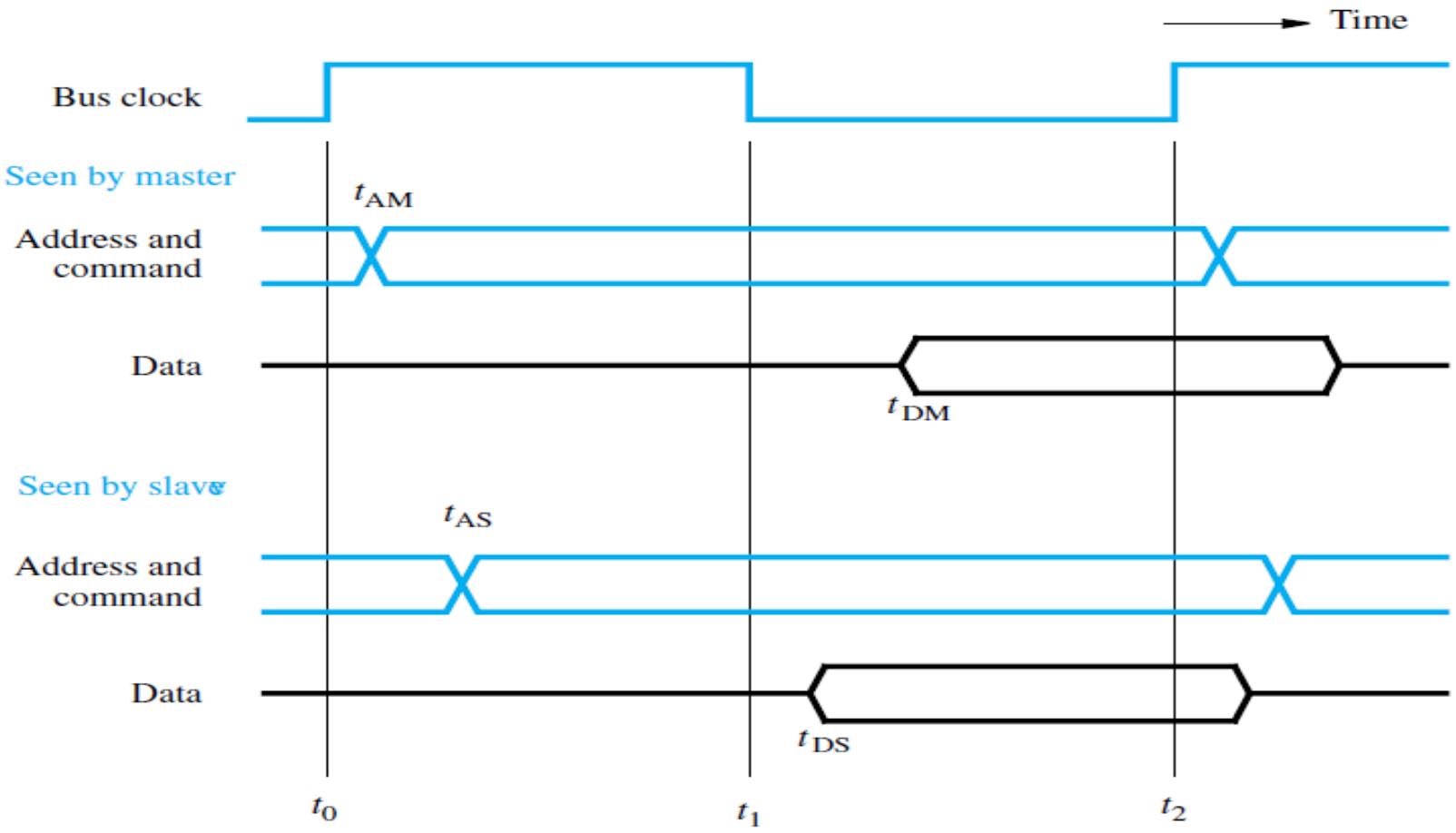
Synchronous Bus: Idealized Timing Diagram (2)

- At time ***t*0**, the master places the device address on the address lines and sends a command on the control lines indicating a Read operation.
- Information travels over the bus.
- The clock pulse width, ***t*1 – *t*0**, must be longer than the maximum **propagation delay** over the bus. Also, it must be long enough to allow all devices to **decode the address** and control signals, so that the addressed device (the slave) can respond at time ***t*1** by placing the requested input data on the data lines.

Synchronous Bus: Idealized Timing Diagram (3)

- At the end of the clock cycle, at time ***t*2**, the master loads the data on the data lines into one of its registers.
- To be loaded correctly into a register, data must be available for a period greater than **the setup time of the register**.
- Hence, the period ***t*2 – *t*1** must be greater than the maximum propagation time on the bus plus the setup time of the master's register.

Synchronous Bus: Input Transfer (1)



Synchronous Bus: Input Transfer (2)

- The master sends the address and command signals on the rising edge of the clock at the beginning of the clock cycle (**at *t*0**). However, these signals do not actually appear on the bus until t_{AM}, largely due to the delay in the **electronic circuit output** from the master to the bus lines.
- A short while later, at t_{AS}, the signals reach the slave. The slave decodes the address, and at ***t*1** sends the requested data.
- Here again, the data signals do not appear on the bus until t_{DS}.
- They travel toward the master and arrive at t_{DM}. At ***t*2**, the master loads the data into its register.
- Hence the period $t2 - t_{DM}$ must be greater than the setup time of that register. The data must continue to be valid after ***t*2** for a period equal to the hold time requirement of the register.

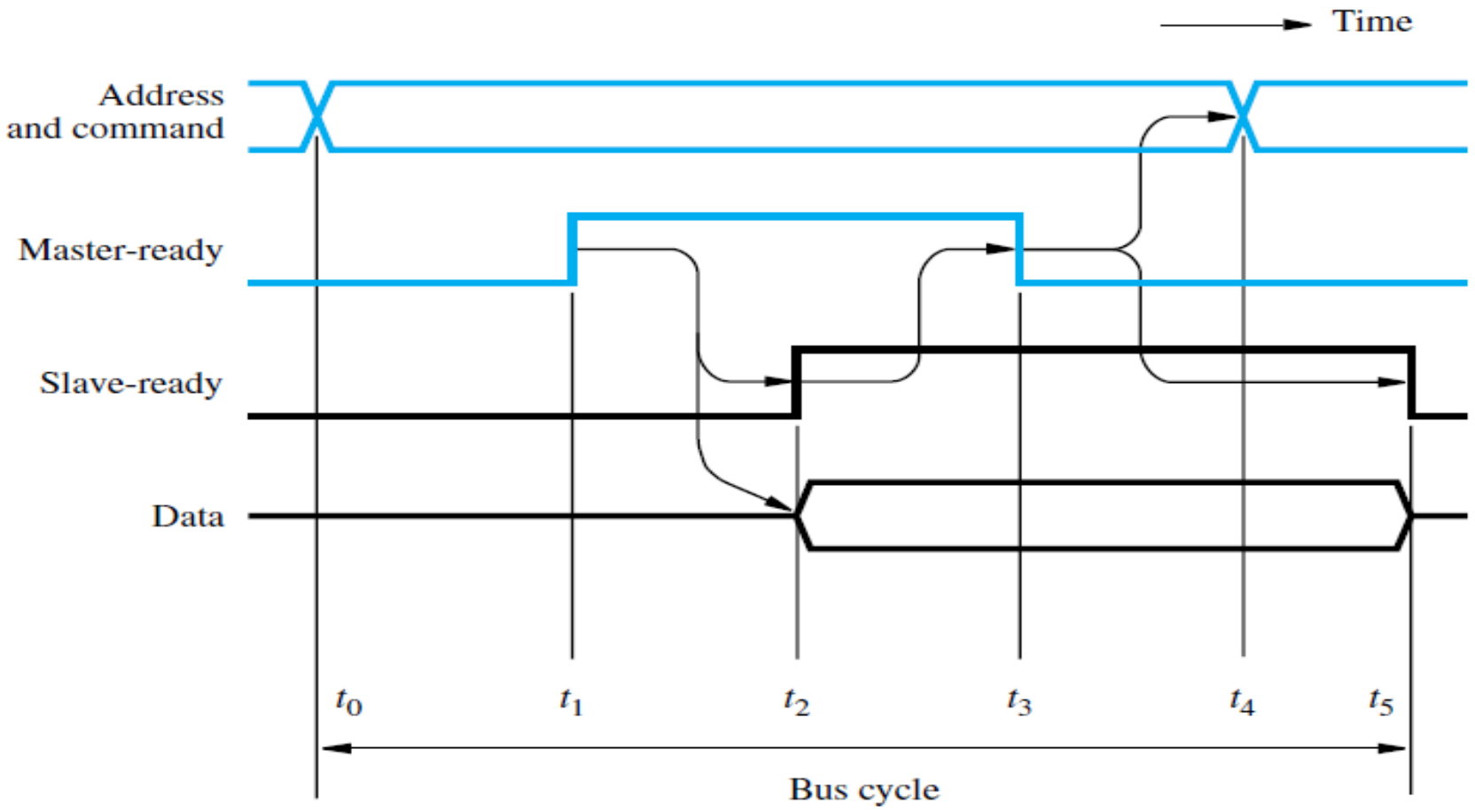
Asynchronous Bus (1)

- An alternative scheme for controlling data transfers on a bus is based on the use of a ***handshake protocol*** between the master and the slave.
- A handshake is an exchange of command and response signals between the master and the slave.

Asynchronous Bus (2)

- The master places the address and command information on the bus. Then it indicates to all devices that it has done so by activating the Master-ready line.
- This causes all devices to decode the address. The selected slave performs the required operation and informs the processor that it has done so by activating the Slave-ready line.
- The master waits for Slave-ready to become asserted before it removes its signals from the bus.
- In the case of a Read operation, it also loads the data into one of its registers.

Asynchronous Bus: Input Transfer (1)



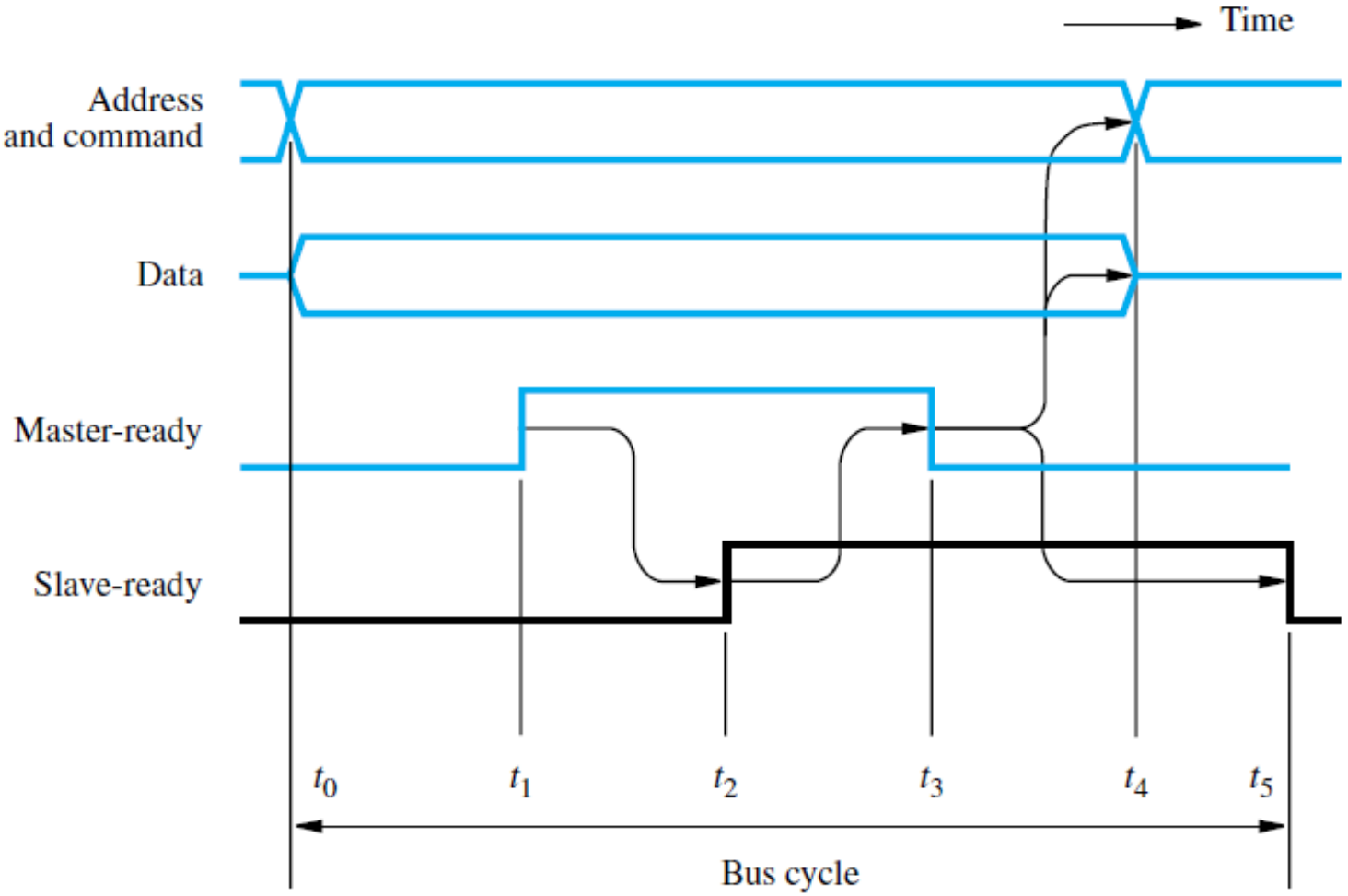
Asynchronous Bus: Input Transfer (2)

- *t*0—The master places the address and command information on the bus, and all devices on the bus decode this information.
- *t*1—The master sets the Master-ready line to 1 to inform the devices that the address and command information is ready. The delay t1-t0 should be longer than the maximum bus skew. And, Sufficient time should be allowed for **the device interface circuitry to decode** the address.
- *t*2—The selected slave performs the required input operation by placing its data on the data lines. At the same time, it sets the Slave-ready signal to 1. If extra delays are introduced by the interface circuitry before it places the data on the bus, the slave must delay the Slave-ready signal accordingly. The period $t2 - t1$ depends on the distance between the master and the slave and on the delays introduced by the **slave's circuitry**.

Asynchronous Bus: Input Transfer (3)

- *t*3—The Slave-ready signal arrives at the master, indicating that the input data are available on the bus. After a **delay to the master loads** the data into its register. Then, it drops the Master-ready signal, indicating that it has received the data.
- *t*4—The master removes the address and command information from the bus. The delay between *t*3 and *t*4 is again intended to **allow for bus skew**. Erroneous addressing may take place if the address, as seen by some device on the bus, starts to change while the Master-ready signal is still equal to 1.
- *t*5—When the device interface receives **the 1-to-0 transition of the Master-ready signal**, it removes the data and the Slave-ready signal from the bus. This completes the input transfer.

Asynchronous Bus: Output Transfer



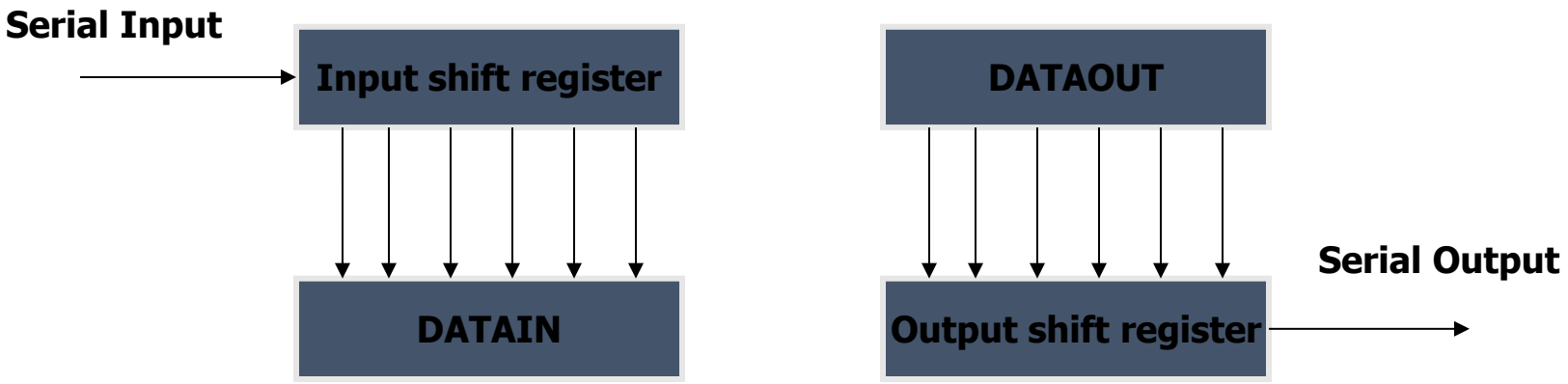
I/O Interface Circuits

- I/O Interface circuits are used to connect I/O devices to a bus. The data path between an interface and an I/O device is called “port”.
  - A parallel port can transfer data of 8 or 16 bits simultaneously to and/or from devices.
  - A serial port transfers data 1 bit at a time.
- A serial port requires fewer wires than a parallel port. Thus, the serial connection is often used with devices that are physically far away from the computer.

Serial-parallel Conversion

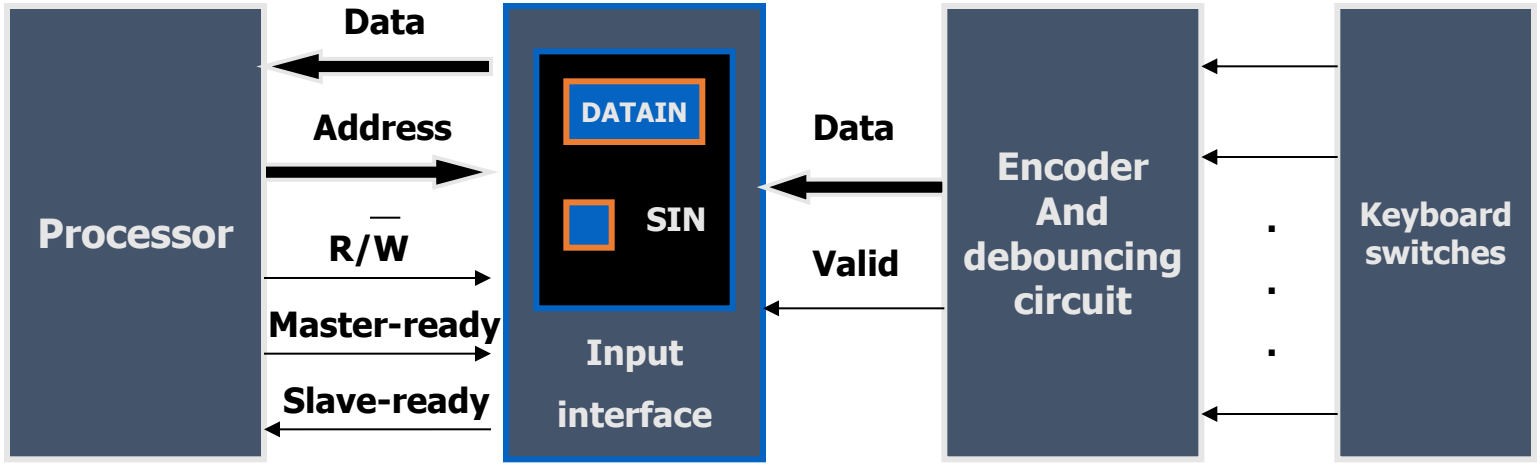
- Serial port: needs conversion between serial and parallel data. It needs to be able to transmit one bit and a time on the device side and 8 bit at a time at the bus side.
- Transformation is achieved using a shift register with parallel access capability.
- Shift register accepts serial bit from an I/O device. when 8 bits have been received the content of the shift register are loaded in parallel into the DATAIN register.

Shift Register



Shift register accepts serial bit from an I/O device. when 8 bits have been received the content of the shift register are loaded in parallel into the DATAIN register.

Input Circuits (1)



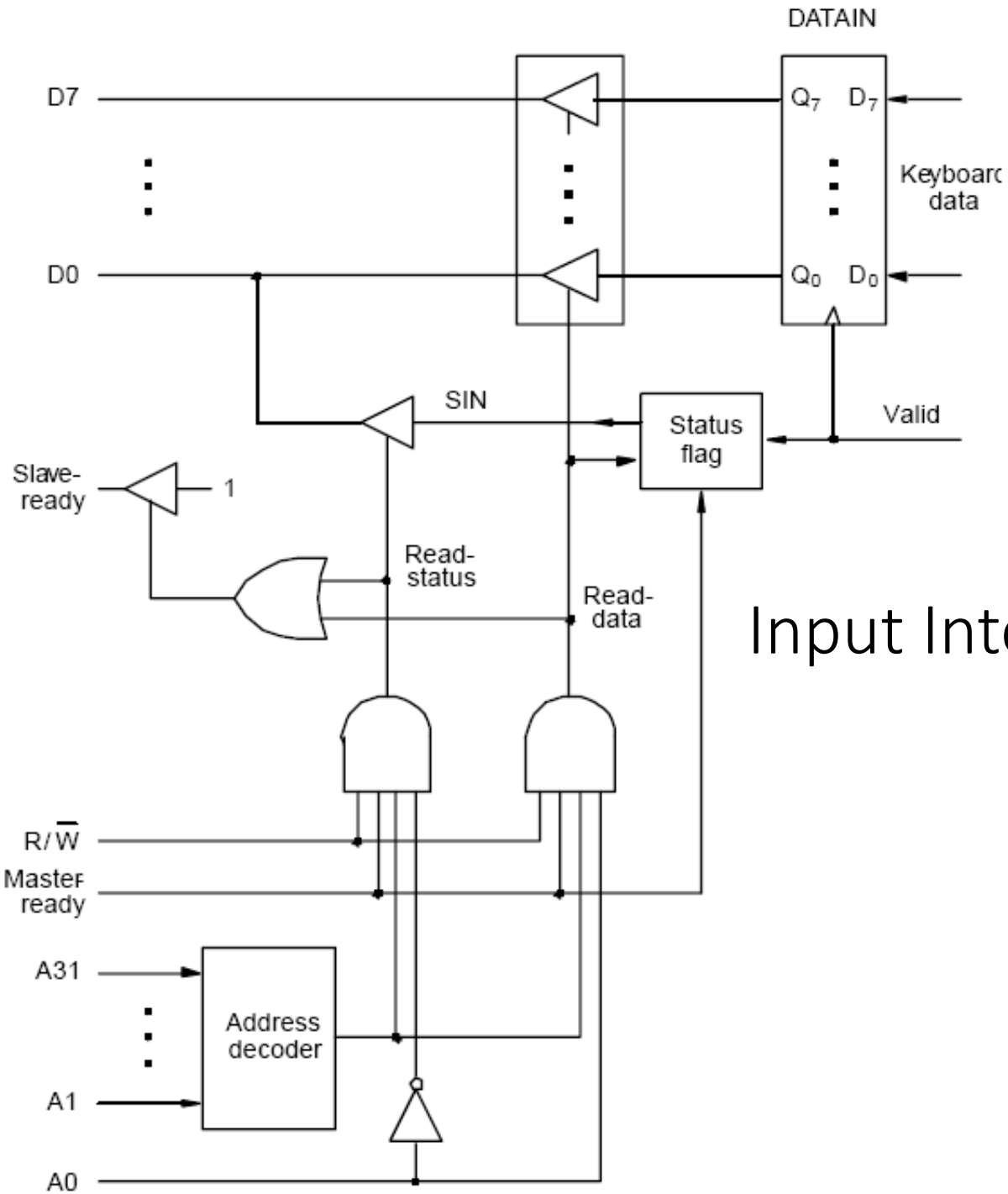
Keyboard to processor connection

Input Circuits (2)

- Keyboard consists of mechanical switches that are opened, when a key is pressed. An ASCII code is generated to a corresponding character.
- De-bouncing circuit is added to prevent a wrongful interpretation that the key is pressed and released several time (cause by the contact bounce of a key).
- Encoder circuit generates a set of bits that represent encoded character and a control signal called valid to indicate a key-press.
- Encoder's output are passed to the interface circuits. When the valid signal goes from 0 to 1, the ASCII code is loaded to the DATAIN register and the SIN bit is set.

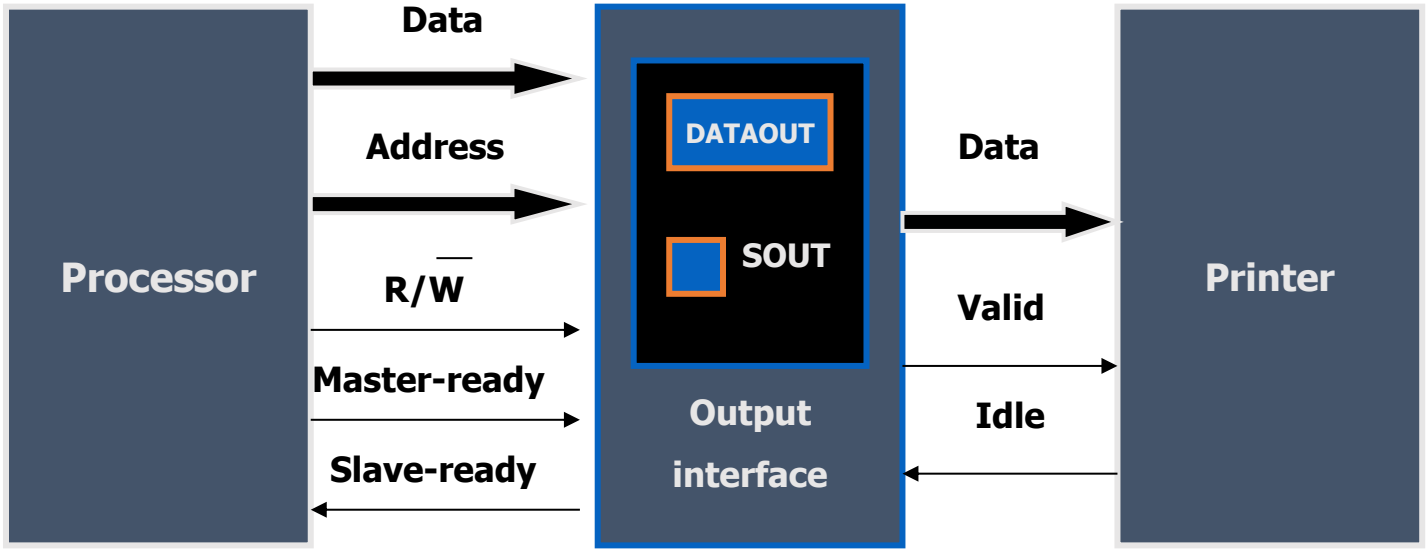
Input Circuits (3)

- The interface circuit is connected to an asynchronous bus where data transfers are controlled using the hand-shake signals (master & slave ready)
  - Master (processor) places address and command onto the bus, master-ready signal is then activated.
  - I/O device decodes the address and activates the slave-ready signal.
  - Read operation is indicated
  - Master removes its signal from the bus after seeing assertion on the slave-ready signal.
  - Master then copies data from the DATAIN register to its own input register.
- Keyboard data is an eight-bit character, where all bits are transferred in parallel.



Input Interface Circuit

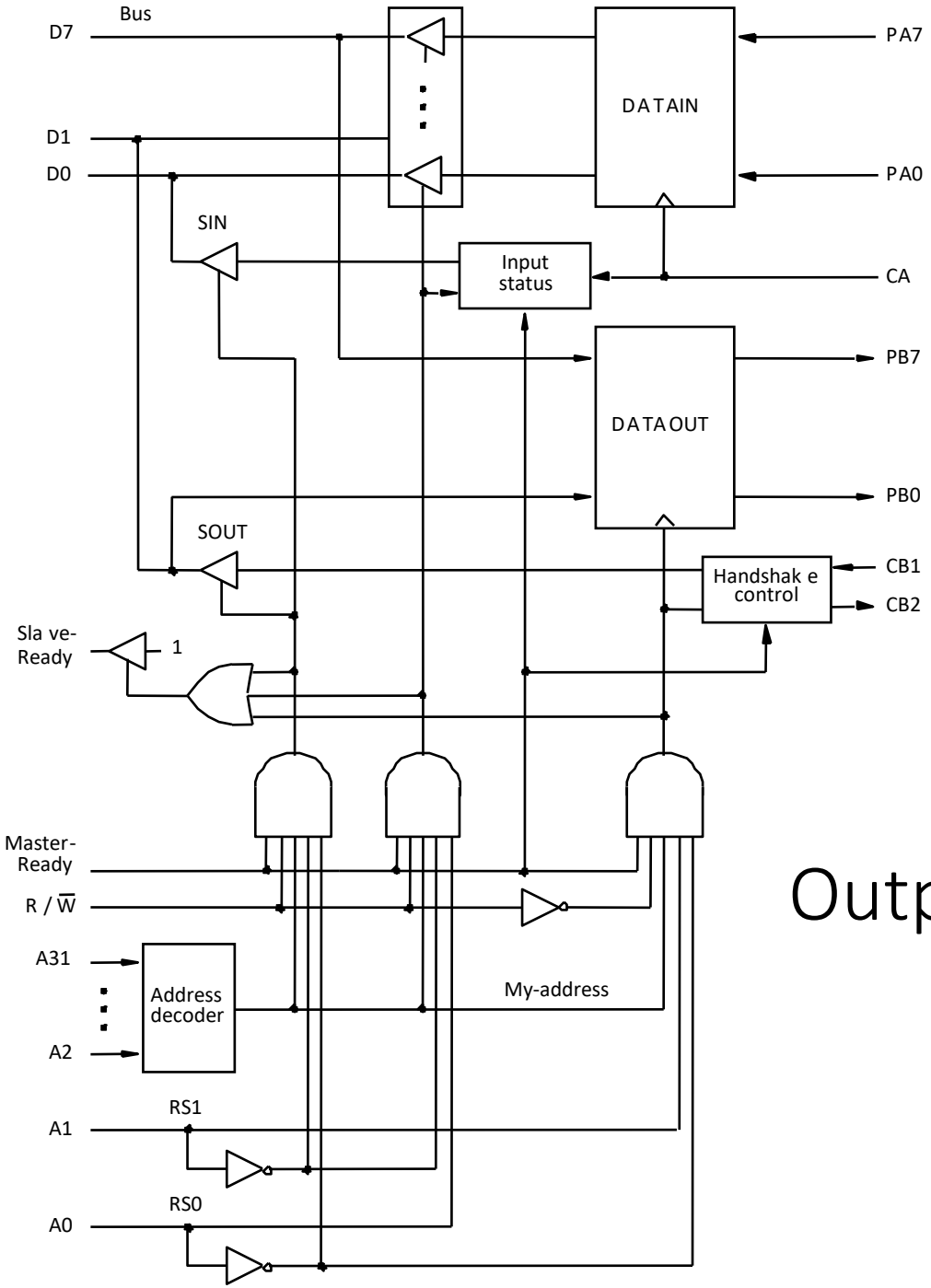
Output Circuits (1)



Printer to processor connection

Output Circuits (2)

- The output circuit is similar to the input circuit
  - Master (processor) places address and data onto the bus, and activates the master-ready signal.
  - The output interface then copies data to its buffer
  - And set the slave-ready signal to notify master that it has done so.
  - Printer (output circuit) asserts the idle signal when it’s ready.
  - Interface circuit places a new character on the data line and activates the valid signal.
  - Printer prints then clears the idle signal, which in turn deactivates the valid signal.

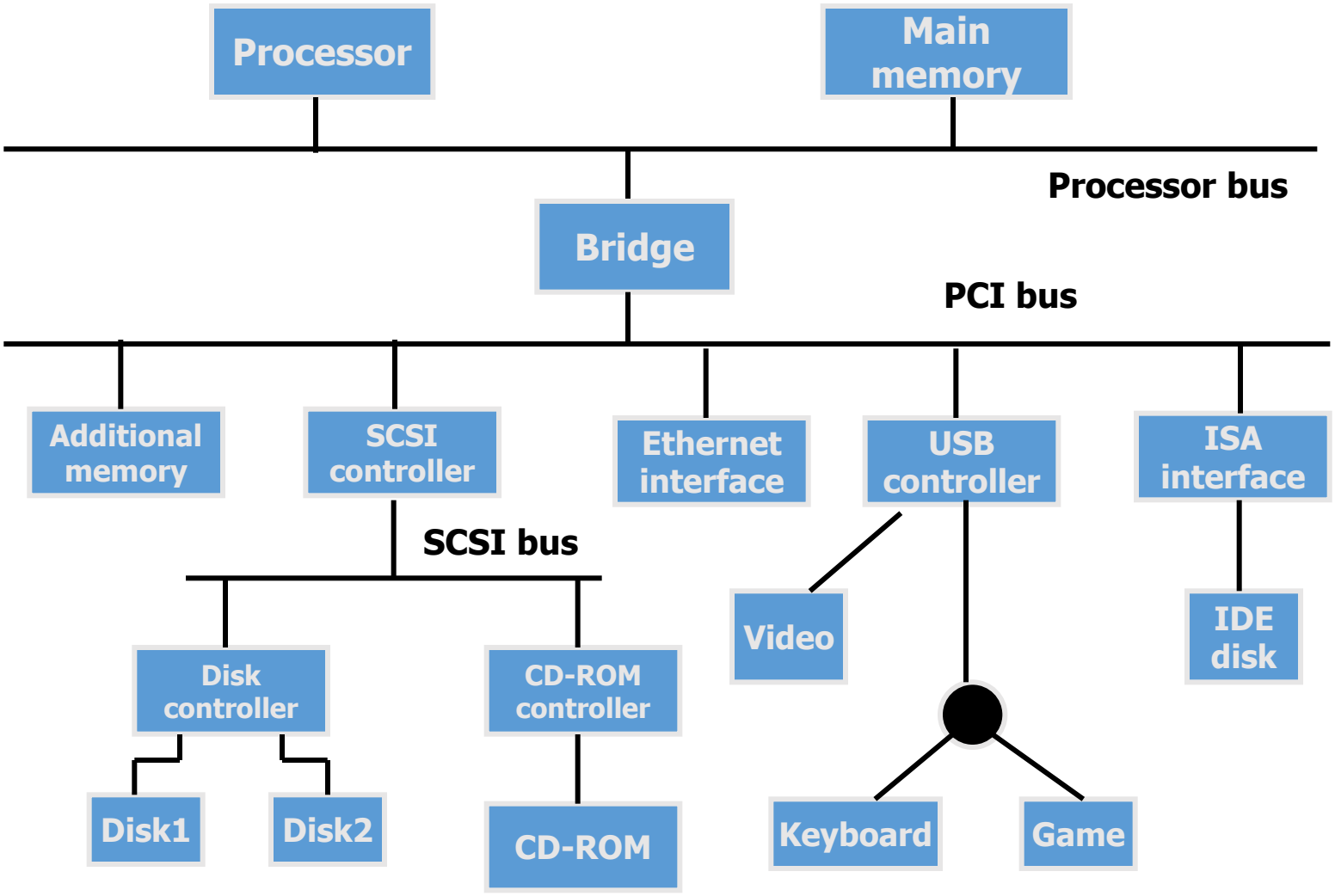


Output Interface Circuit

Expansion Bus

- A computer has a printed circuit board (mother board) that houses the processor chip, RAM, and some I/O interfaces. Also it has a few connectors where additional interfaces can be plugged.
- Some standards have been designed for this expansion bus (path way between devices and a processor)
- Expansion bus is connected to the processor bus through a circuit called bridge.

Standard I/O interface Structure



An example of a computer system using different interface standards

Standard I/O interface (1)

- Three widely used expansion bus standards are:
  - PCI (Peripheral Component Interconnect)
  - SCSI (Small Computer System Interface)
  - USB (Universal Serial Bus)
- Devices connected to these expansion buses appear to a processor as if they were connected directly to the processor bus. There is only a small amount of delay introduced by bridge circuit.

Standard I/O interface (2)

- SCSI bus is a high speed parallel bus.
- USB uses serial transmission for some equipment.
- ISA is an earlier standard that is substantially slower than PCI (slow hand-shake protocol).
- Some computers still have ISA connectors to be compatible with devices such as IDE disk.

Supplementary Section

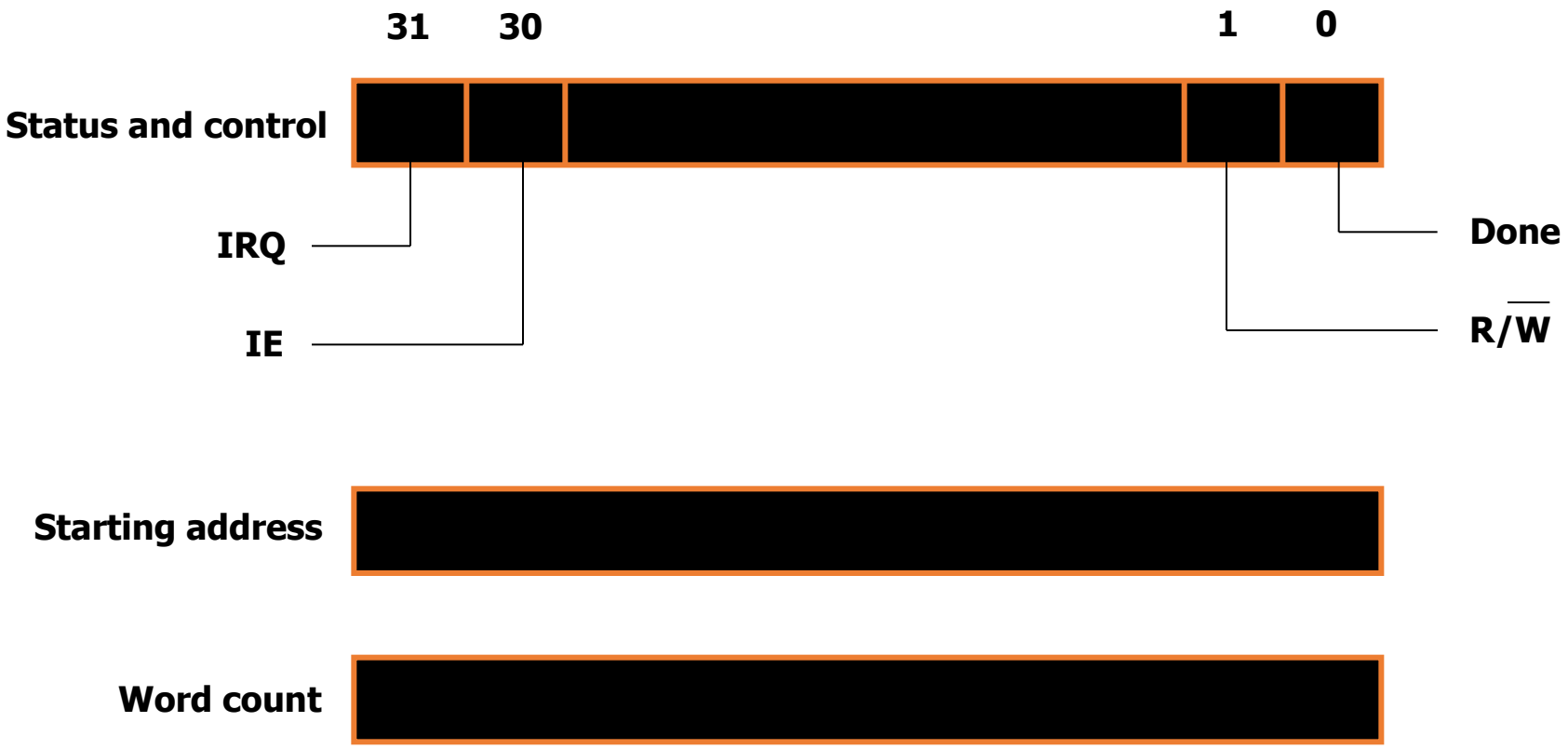
Direct Memory Access (1)

- Direct Memory Access (DMA) is an alternative approach to transfer a large block of data in high speed between an external device and the main memory.
- DMA is performed by a control circuit that is a part of an I/O interface. The DMA controller performs a processor's function when accessing a memory.

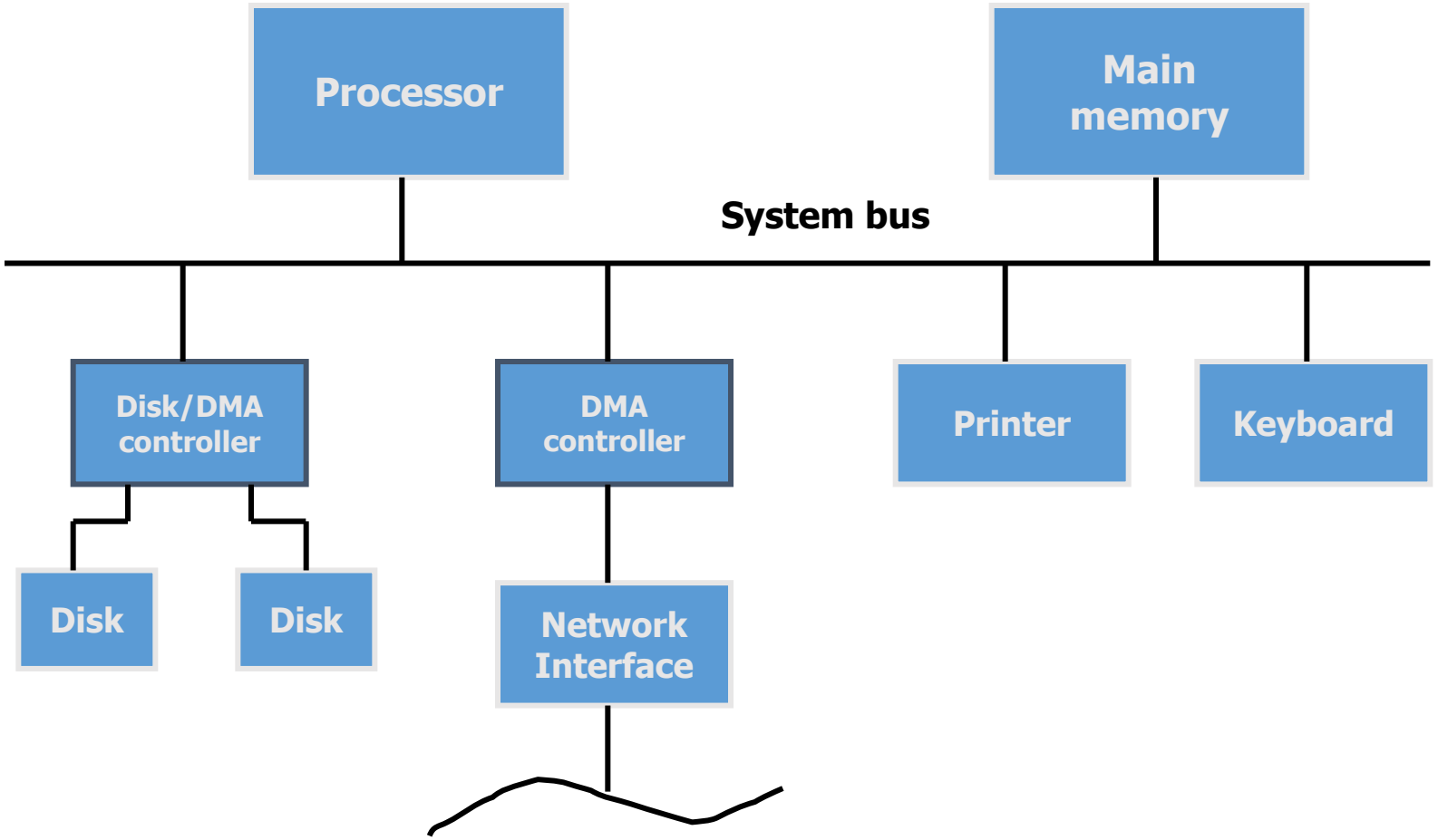
Direct Memory Access (2)

- To initiate a block transfer, the program executing on a processor sends the starting address, a number of words in block, and a direction of transfer to the DMA controller. The DMA controller then proceeds with the transfer. When finished a processor is notified by an interrupt.
- While DMA is taking place, the program that requested the transfer cannot continue, and the processor can be used by another program.

Register in DMA Interface



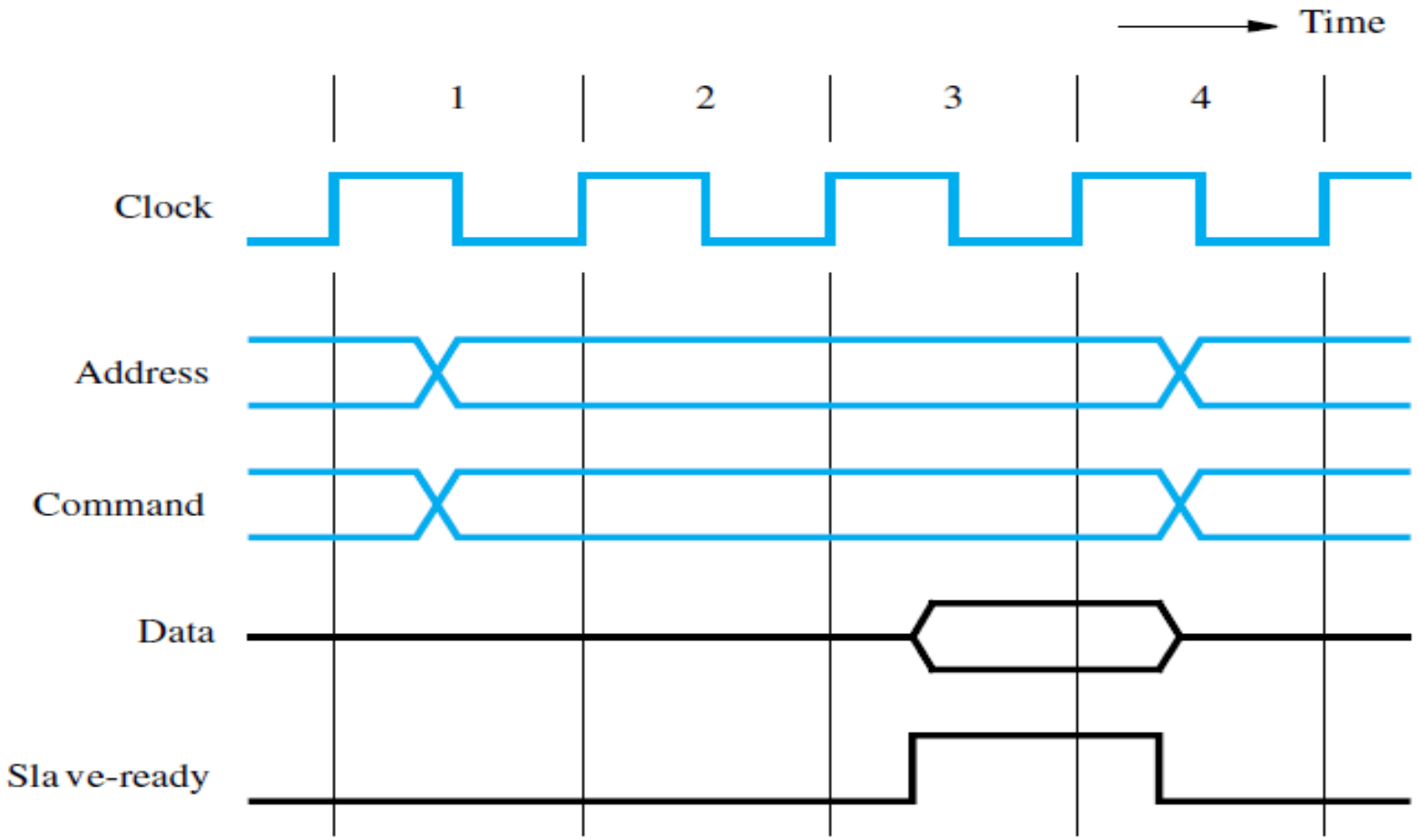
DMA Architecture



Synchronous Bus: Multicycle Data Transfer (1)

- However, it has some limitations.
- Because a transfer has to be completed within one clock cycle, the clock period, $t2 - t0$, must be chosen to accommodate the longest delays on the bus and the slowest device interface.
- This forces all devices to operate at the speed of the slowest device.
- To overcome these limitations, most buses incorporate control signals that represent a **response from the device**. These signals inform the master that the slave has recognized its address and that it is ready to participate in a data transfer operation. They also make it possible to adjust the duration of the data transfer period to match the response speeds of different devices.
- This is often accomplished by allowing a complete data transfer operation to span several clock cycles. Then, the number of clock cycles involved can vary from one device to another.

Synchronous Bus: Multicycle Data Transfer (2)



Synchronous Bus: Multicycle Data Transfer (3)

- During clock cycle 1, the master sends address and command information on the bus, requesting a Read operation.
- The slave receives this information and decodes it. It begins to access the requested data on the active edge of the clock at the beginning of clock cycle 2.
- Due to the delay involved in getting the data, the slave cannot respond immediately. The data become ready and are placed on the bus during clock cycle 3.
- The slave asserts a control signal called Slave-ready at the same time.
- The master, which has been waiting for this signal, loads the data into its register at the end of the clock cycle.
- The slave removes its data signals from the bus and returns its Slave-ready signal to the low level at the end of cycle 3.
- The bus transfer operation is now complete, and the master may send new address and command signals to start a new transfer in clock cycle 4.
- If the addressed device does not respond at all, the master waits for some predefined maximum number of clock cycles, then aborts the operation.

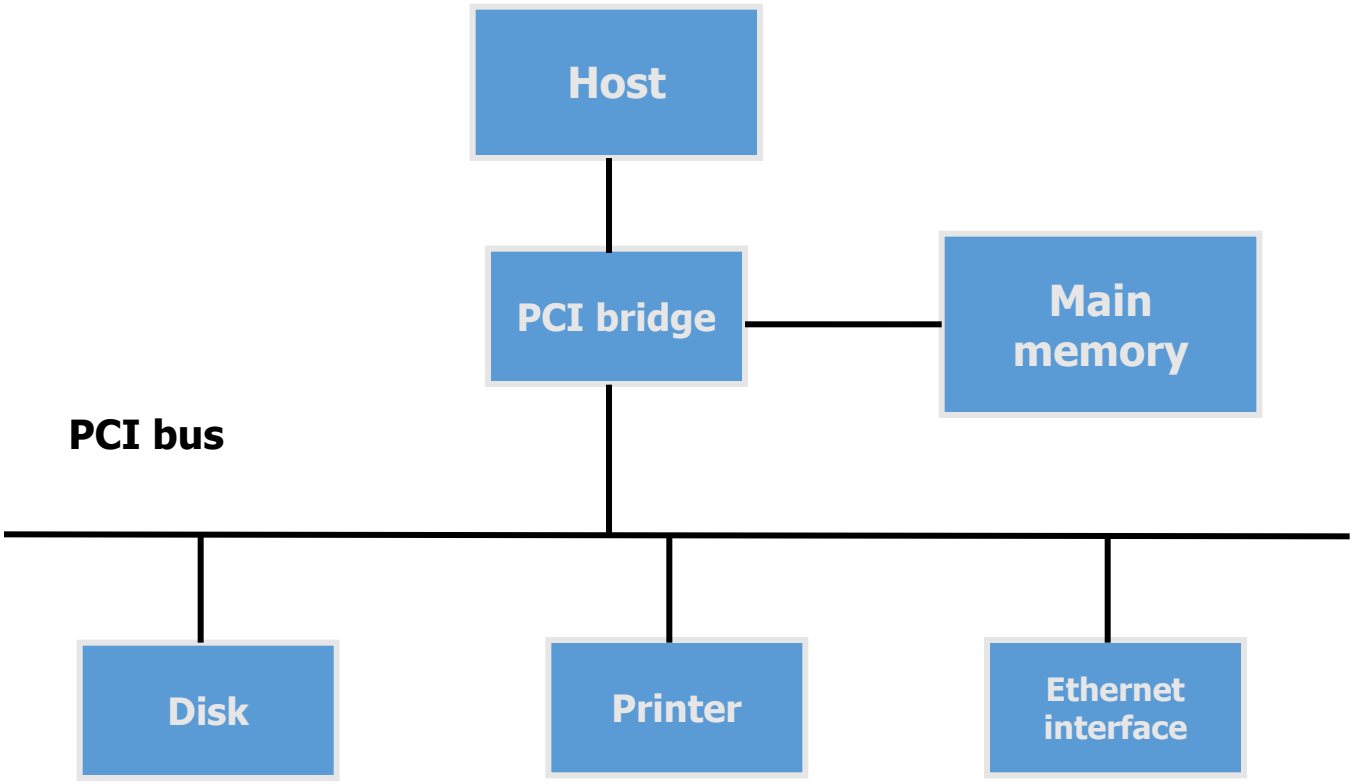
PCI BUS (1)

- PCI bus is processor independent. Devices connected to the PCI bus are assigned with addresses in the memory (as if they are directly connected to a processor bus).
- Advantages
  - processor independent
  - low cost
  - plug-and-play

PCI BUS (2)

- Data transfer is done in a burst of words. The bus supports 3 independent address spaces: memory, I/O, and configuration spaces.
  - The I/O space is intended for a processor that has a separate I/O address space.
  - The configuration space is used to support plug and play.
- In data transfer operation, the address will be accompanied with a 4-bit command to identify which address space to be used.

PCI Bus Structure



Use of a PCI bus in a computer system

PCI Bus (3)

- Address information is needed in the bus only until **the slave device is selected**. The device stores the address locally. This allow the address lines to be used for sending the **next block of data** in the next clock.
- At any given time, one device is the bus master that has the right to **initiate data transfer** (called initiator). The initiator is usually the processor or DMA controller. The addressed device that responds to read and write commands is called ‘target’.
- When an I/O device is connected to a computer, actions are needed to configure both the device and the software that **communicates** with the device.

PCI Bus (4)

- PCI incorporates, in each I/O device interface, a small configuration ROM that stores information about the device. **Configuration ROMs** are accessible in the configuration address space.
- The software reads these ROMs when the system is **powered up** to determine the type of the device that is connected to PCI. The system also learns all the device's options.
- When initializing, devices are assigned with the address in the I/O address space. The addresses are then written on the **device register**.
- The user does not have to set jumpers/switches just plug in the device and install the driver.

SCSI Bus (1)

- A narrow SCSI has 8 data lines and can transfer data one byte at a time.
- A wide SCSI has 16 data lines.
- Transfer rate varies between 5-320 Mbytes/sec.
- Higher rate can be achieved for shorter length cable and fewer devices.
- Devices connected to the SCSI bus are not a part of the address space of a processor. Devices are connected to the processor bus via a **SCSI controller**.

SCSI Bus(2)

- DMA is used to transfer data between memory and devices.
- SCSI allows **overlapping** of data transfer request.
- Processor and SCSI controller do not need to be aware of the detail operations of any particular device (device independent).
- Handshake signaling is used to control information transfer (the same way as parallel port).

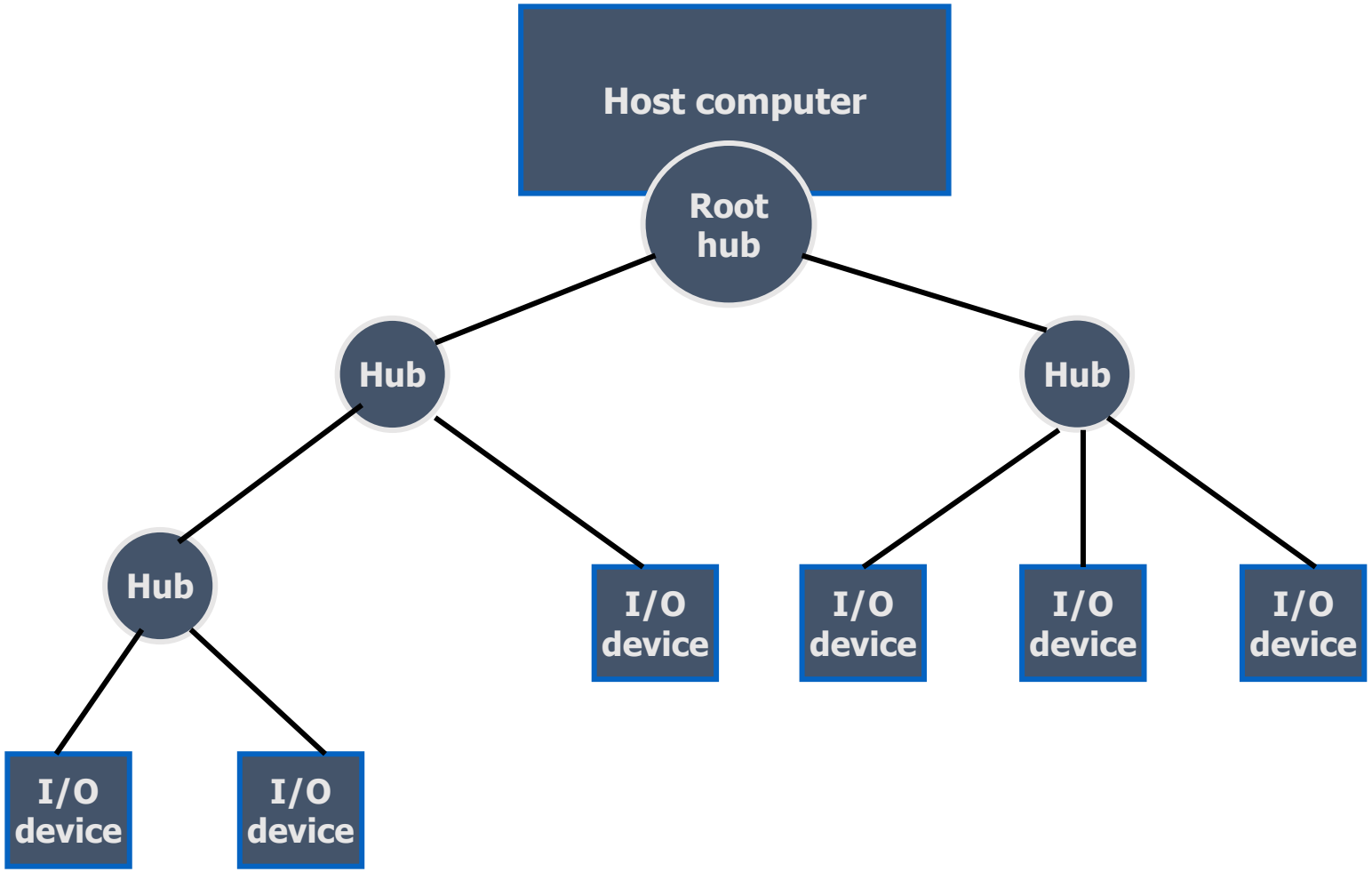
Universal Serial Bus (1)

- USB is a simple, low-cost mechanism to connect all kinds of devices to a computer (wired, wireless, microphone, digital camera).
- USB supports
  - low-speed (1.5 Mbits/s)
  - high-speed (12 Mbits/s)
  - super high speed (480 Mbits/s) introduced in USB2.0
- USB accommodates a wide range of data transfer characteristics for I/O devices including telephone and the Internet.

Universal Serial Bus (2)

- Plug and play: devices can be connected while the system is in operation. The device driver is identified and installed automatically.
- USB uses **serial transmission** to satisfy a low cost constraint. Clock and data are encoded together and transmitted as a single signal.

Universal Serial Bus Structure



Universal Serial Bus tree structure

USB Hub

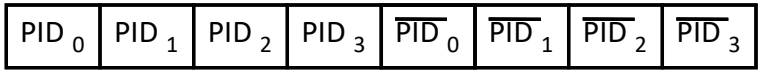
- Hubs are an intermediate control point between a host and I/O devices. The root node connects the entire tree. The leave nodes are I/O devices. With this tree structure, many devices can be connected using a point to point serial link.
- During operations, a hub copies a message from upstream connection to all its downstream ports. A message sent from host is broadcast to all devices, but only the addressed device will respond to the message.

USB Operation

- USB operates on the basis of polling. A device can send a message only in respond to a polled message from a host. There can be no conflict for upstream transmission.
- USB root hub is attached to the processor bus, where it appears as a single device. The root hub will forward all messages downstream. Thus each hub and device needs an address that is local to the USB tree (7-bits). These addresses are assigned arbitrarily once the device is plugged in.

USB Protocols

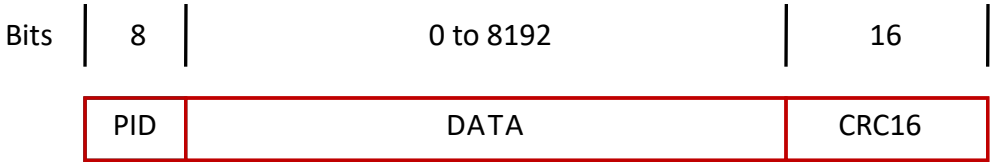
- The information transferred on the USB can be divided into two broad categories: control and data.
  - Control packets perform such tasks as addressing a device to initiate data transfer, acknowledging that data have been received correctly, or indicating an error.
  - Data packets carry information that is delivered to a device.
- A packet consists of one or more fields containing different kinds of information. The first field of any packet is called the packet identifier, PID, which identifies the type of that packet.



(a) Packet identifier field

(b) Token packet, IN or OUT

Control packets used for controlling data transfer operations are called token packets.



(c) Data packet

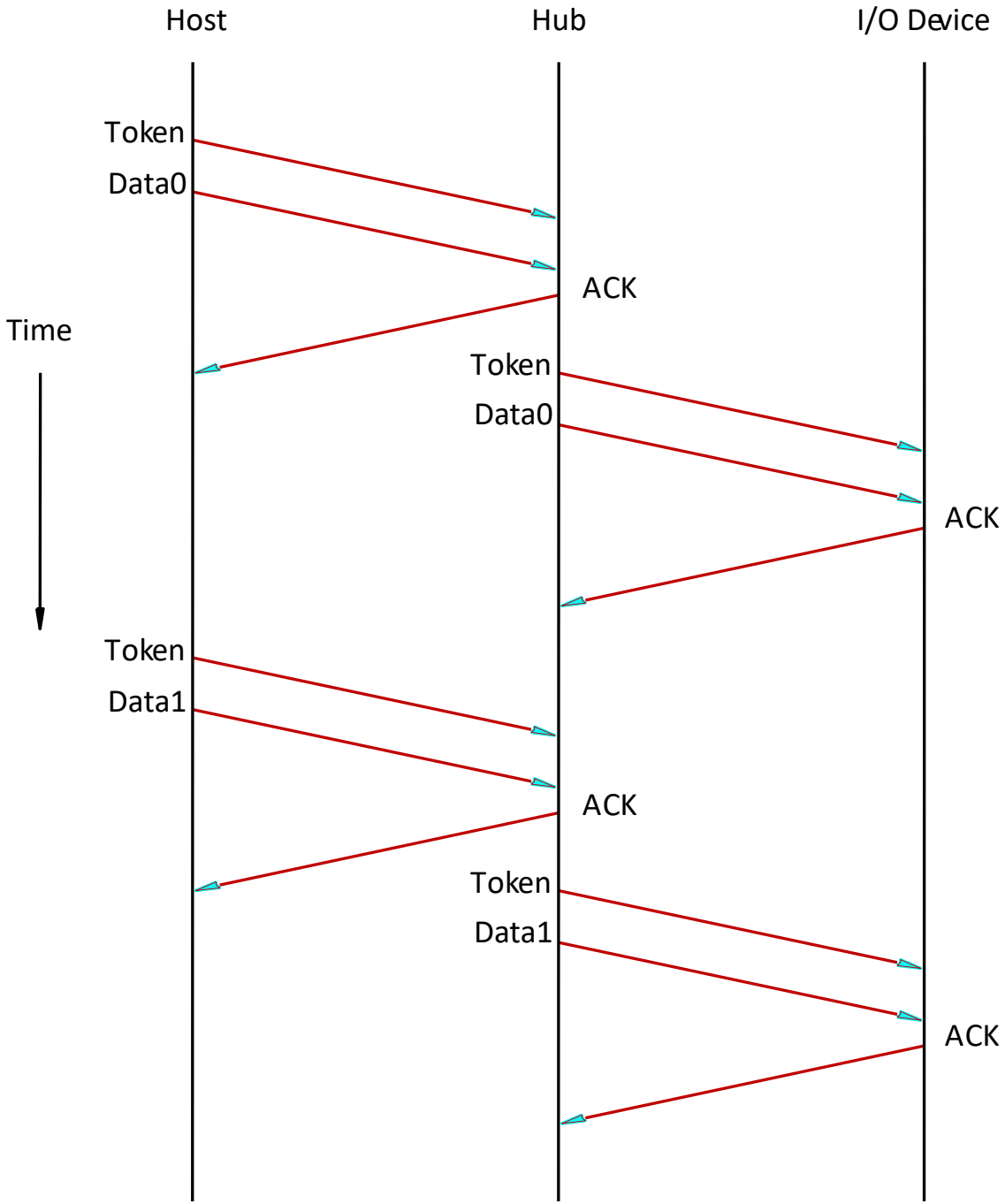


Figure: An output transfer